SQL FOR DATA ANALYTICS

Module 4 Assessment | Assessment Code: M4-A1

Program: Data Analytics | Stack: MySQL | Total Time: 3 Hours

Student Name: _____

Roll Number: _____

Submission Date: _____

Section A — Concept Application

A1. GROUP BY and HAVING for Restaurant Revenue

I would filter the orders to the previous calendar month, group the remaining records by restaurant, and calculate each restaurant's revenue and order count. HAVING is used to filter groups after aggregation, so it can test SUM(total_amount) > 100000 and COUNT(*) >= 50. WHERE filters individual rows before grouping and cannot directly filter aggregate results.

```
SELECT restaurant_name,
       SUM(total_amount) AS total_revenue,
       COUNT(*) AS order_count
FROM delivery_orders
WHERE order_date >= DATE_FORMAT(CURRENT_DATE - INTERVAL 1 MONTH, '%Y-%m-01')
  AND order_date <  DATE_FORMAT(CURRENT_DATE, '%Y-%m-01')
  AND order_status = 'Completed'
GROUP BY restaurant_name
HAVING SUM(total_amount) > 100000
   AND COUNT(*) >= 50;
```

Replacing HAVING with WHERE for the aggregate conditions would cause an SQL error because aggregate functions such as SUM and COUNT are evaluated after the row-filtering stage. The date and status conditions correctly belong in WHERE.

A2. Include Customers with No Orders

Use LEFT JOIN from customers to orders. It preserves every customer row and returns NULL for order columns when there is no matching order. INNER JOIN only returns customers with matching orders, so customers who have never ordered would disappear. A ROW_NUMBER() subquery can select each customer's most recent order.

```
WITH ranked_orders AS (
    SELECT o.*,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_date DESC, order_id DESC
           ) AS rn
    FROM orders AS o
)
SELECT c.customer_id, c.name AS customer_name,
       ro.order_id, ro.order_date, ro.amount
FROM customers AS c
LEFT JOIN ranked_orders AS ro
       ON c.customer_id = ro.customer_id
      AND ro.rn = 1;
```

A3. Scalar Subquery for Faster-than-Average Agents

A scalar subquery is appropriate because the overall platform average returns one value. Compare each agent's average delivery time with that value in WHERE. Placing a scalar subquery in SELECT is also valid if it returns one value; it would display the overall average as a column, but would not by itself filter agents.

```
SELECT agent_name,
       AVG(delivery_time_mins) AS avg_delivery_time
FROM deliveries
WHERE delivery_time_mins IS NOT NULL
GROUP BY agent_name
HAVING AVG(delivery_time_mins) < (
    SELECT AVG(delivery_time_mins)
    FROM deliveries
    WHERE delivery_time_mins IS NOT NULL
);
```

A4. RANK() versus DENSE_RANK()

Both functions assign the same rank to tied revenue values. RANK() leaves gaps after ties, while DENSE_RANK() does not. For a report that should include the top three distinct revenue levels, I would choose DENSE_RANK(). If the business requires competition-style ranking positions, RANK() is appropriate.

Restaurant	Revenue	RANK()	DENSE_RANK()
A	₹20,000	1	1
B	₹20,000	1	1
C	₹15,000	3	2

```
SELECT city, restaurant_name, total_revenue,
       RANK() OVER (PARTITION BY city ORDER BY total_revenue DESC) AS revenue_rank,
       DENSE_RANK() OVER (PARTITION BY city ORDER BY total_revenue DESC) AS dense_revenue_rank
FROM restaurant_revenue;
```

With RANK(), the third restaurant is ranked 3, so filtering rank <= 3 returns the tied first-place restaurants and the third-place restaurant. With DENSE_RANK(), the third restaurant is rank 2, so rank <= 3 can include restaurants from three distinct revenue levels.

A5. CASE WHEN Customer Segmentation

CASE WHEN creates a classification in the query output without creating a new table. Since Platinum is defined as spending above ₹5,000, exactly ₹5,000 belongs to Gold. The conditions should be ordered and written with clear inclusive/exclusive boundaries.

```
SELECT customer_id, customer_name, total_spend,
       CASE
           WHEN total_spend > 5000 THEN 'Platinum'
           WHEN total_spend >= 2000 THEN 'Gold'
           ELSE 'Standard'
       END AS customer_tier
FROM customer_spending;
```

A6. Indexes and Performance Trade-offs

An index is a data structure (commonly a B-tree in MySQL) that helps the database locate rows without scanning an entire table. An index on frequently used JOIN keys can reduce rows/pages examined and make joins faster, especially when the indexed columns are selective. Indexes are not always beneficial: they consume disk and memory, add overhead to INSERT/UPDATE/DELETE, and may be ignored by the optimizer when a query reads a large portion of the table. Indexing every column increases storage and write costs. Check EXPLAIN plans and measure query performance before and after adding a suitable index.

```
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
EXPLAIN
SELECT c.name, COUNT(o.order_id)
FROM customers AS c
JOIN orders AS o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name;
```

Section B — Practical Coding Tasks

Task 1: Menu & Order Explorer

Create the table and insert 8 sample rows across Mumbai, Pune, and Ahmedabad.

```sql
CREATE DATABASE IF NOT EXISTS food_delivery_analytics;
USE food_delivery_analytics;

CREATE TABLE food_orders (
    order_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    restaurant_name VARCHAR(100) NOT NULL,
    item_ordered VARCHAR(100) NOT NULL,
    quantity INT NOT NULL,
    price_per_item DECIMAL(10,2) NOT NULL,
    order_date DATE NOT NULL,
    city VARCHAR(60) NOT NULL
);

INSERT INTO food_orders VALUES
(1,'Aarav Shah','Spice Route','Paneer Tikka',2,280,'2026-09-02','Mumbai'),
(2,'Diya Patel','Burger Hub','Double Burger',1,350,'2026-09-03','Pune'),
(3,'Rohan Mehta','Coastal Curry','Fish Thali',2,450,'2026-09-04','Mumbai'),
(4,'Sara Khan','Pizza Point','Farmhouse Pizza',1,520,'2026-09-05','Ahmedabad'),
(5,'Kabir Joshi','Spice Route','Veg Biryani',3,220,'2026-09-06','Mumbai'),
(6,'Anaya Desai','Cafe Bloom','Pasta',2,310,'2026-09-07','Pune'),
(7,'Ishaan Rao','Coastal Curry','Prawn Curry',1,650,'2026-09-08','Mumbai'),
(8,'Mira Shah','Pizza Point','Cheese Burst Pizza',2,480,'2026-09-09','Ahmedabad');
```

B1(a). Mumbai orders sorted by price per item, highest first

```sql
SELECT *
FROM food_orders
WHERE city = 'Mumbai'
ORDER BY price_per_item DESC;
```

B1(b). Distinct restaurants with orders

```sql
SELECT DISTINCT restaurant_name
FROM food_orders
ORDER BY restaurant_name;
```

B1(c). Top 5 most expensive individual orders

This interprets order value as quantity multiplied by price per item.

```sql
SELECT order_id, customer_name, restaurant_name, item_ordered,
       quantity, price_per_item,
       quantity * price_per_item AS order_total
FROM food_orders
ORDER BY order_total DESC
LIMIT 5;
```

Task 2: Restaurant Revenue Summary

```
CREATE TABLE delivery_orders (
    order_id INT PRIMARY KEY,
    restaurant_name VARCHAR(100) NOT NULL,
    city VARCHAR(60) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    order_status VARCHAR(30) NOT NULL,
    order_date DATE NOT NULL
);

INSERT INTO delivery_orders VALUES
(101,'Spice Route','Mumbai',1800,'Completed','2026-09-01'),
(102,'Spice Route','Mumbai',2200,'Completed','2026-09-02'),
(103,'Spice Route','Mumbai',1600,'Completed','2026-09-03'),
(104,'Spice Route','Mumbai',2500,'Completed','2026-09-04'),
(105,'Burger Hub','Pune',1200,'Completed','2026-09-05'),
(106,'Burger Hub','Pune',900,'Completed','2026-09-06'),
(107,'Burger Hub','Pune',1400,'Completed','2026-09-07'),
(108,'Coastal Curry','Mumbai',4200,'Completed','2026-09-08'),
(109,'Coastal Curry','Mumbai',3800,'Completed','2026-09-09'),
(110,'Pizza Point','Ahmedabad',6000,'Completed','2026-09-10'),
(111,'Pizza Point','Ahmedabad',5200,'Completed','2026-09-11'),
(112,'Cafe Bloom','Pune',750,'Completed','2026-09-12');
```

B2(a–c). Aggregation, HAVING, and CASE classification

```
SELECT restaurant_name,
       SUM(total_amount) AS total_revenue,
       COUNT(*) AS total_orders,
       ROUND(AVG(total_amount), 2) AS average_order_value,
       CASE
           WHEN SUM(total_amount) > 10000 THEN 'High Revenue'
           ELSE 'Moderate Revenue'
       END AS revenue_category
FROM delivery_orders
WHERE order_status = 'Completed'
GROUP BY restaurant_name
HAVING SUM(total_amount) > 5000
ORDER BY total_revenue DESC;
```

The query uses completed orders for revenue reporting. If the assessment expects all statuses to count, remove the WHERE status filter. The HAVING clause keeps restaurants whose total revenue exceeds ₹5,000.

Task 3: Customer–Order–Restaurant Joined Report

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(60) NOT NULL,
    phone VARCHAR(20)
);

CREATE TABLE restaurants (
    restaurant_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    cuisine_type VARCHAR(60),
    city VARCHAR(60) NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    restaurant_id INT,
    amount DECIMAL(10,2) NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

INSERT INTO customers VALUES
(1,'Aarav Shah','Mumbai','9000000001'),
(2,'Diya Patel','Pune','9000000002'),
(3,'Rohan Mehta','Mumbai','9000000003'),
(4,'Sara Khan','Ahmedabad','9000000004'),
(5,'Kabir Joshi','Pune','9000000005'),
(6,'Anaya Desai','Ahmedabad','9000000006');

INSERT INTO restaurants VALUES
(1,'Spice Route','Indian','Mumbai'),
(2,'Burger Hub','Fast Food','Pune'),
(3,'Coastal Curry','Seafood','Mumbai'),
(4,'Pizza Point','Italian','Ahmedabad'),
(5,'Cafe Bloom','Cafe','Pune'),
(6,'Tandoori House','Indian','Ahmedabad');

INSERT INTO orders VALUES
(201,1,1,800,'2026-09-01'),
(202,2,2,650,'2026-09-02'),
(203,3,3,1250,'2026-09-03'),
(204,4,4,900,'2026-09-04'),
(205,1,3,700,'2026-09-05'),
(206,5,5,450,'2026-09-06');
```

B3(a). INNER JOIN order report

```
SELECT o.order_id, c.name AS customer_name,
       r.name AS restaurant_name, o.amount, o.order_date
FROM orders AS o
INNER JOIN customers AS c ON o.customer_id = c.customer_id
INNER JOIN restaurants AS r ON o.restaurant_id = r.restaurant_id
ORDER BY o.order_date;
```

B3(b). LEFT JOIN to include customers with no orders

```
SELECT c.customer_id, c.name AS customer_name, c.city,
       o.order_id, o.amount, o.order_date
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
ORDER BY c.customer_id, o.order_date;
```

B3(c). Orders where customer and restaurant are in the same city

```
SELECT o.order_id, c.name AS customer_name, c.city AS customer_city,
       r.name AS restaurant_name, r.city AS restaurant_city, o.amount
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
JOIN restaurants AS r ON o.restaurant_id = r.restaurant_id
WHERE c.city = r.city;
```

Task 4: Delivery Agent Performance Ranking

```
CREATE TABLE deliveries (
    delivery_id INT PRIMARY KEY,
    agent_name VARCHAR(100) NOT NULL,
    restaurant_name VARCHAR(100) NOT NULL,
    delivery_time_mins INT NULL,
    order_date DATE NOT NULL
);

INSERT INTO deliveries VALUES
(1,'Neel','Spice Route',28,'2026-09-01'),
(2,'Priya','Burger Hub',35,'2026-09-01'),
(3,'Neel','Coastal Curry',32,'2026-09-02'),
(4,'Amit','Pizza Point',42,'2026-09-02'),
(5,'Priya','Cafe Bloom',31,'2026-09-03'),
(6,'Zoya','Spice Route',25,'2026-09-03'),
(7,'Amit','Coastal Curry',39,'2026-09-04'),
(8,'Zoya','Burger Hub',29,'2026-09-04'),
(9,'Neel','Pizza Point',30,'2026-09-05'),
(10,'Priya','Spice Route',34,'2026-09-05'),
(11,'Amit','Cafe Bloom',45,'2026-09-06'),
(12,'Zoya','Coastal Curry',27,'2026-09-06');
```

```
WITH agent_stats AS (
    SELECT agent_name,
           AVG(delivery_time_mins) AS avg_delivery_time
    FROM deliveries
    WHERE delivery_time_mins IS NOT NULL
    GROUP BY agent_name
)
SELECT agent_name,
       ROUND(avg_delivery_time, 2) AS avg_delivery_time,
       RANK() OVER (ORDER BY avg_delivery_time ASC) AS speed_rank,
       (SELECT AVG(delivery_time_mins)
        FROM deliveries
        WHERE delivery_time_mins IS NOT NULL) AS platform_avg,
       CASE
           WHEN avg_delivery_time <= (
               SELECT AVG(delivery_time_mins)
               FROM deliveries
               WHERE delivery_time_mins IS NOT NULL
           ) THEN 'On Track'
           ELSE 'Needs Improvement'
       END AS performance_status
FROM agent_stats
ORDER BY speed_rank, agent_name;
```

Section C — Mini Capstone Project

The capstone reuses the customers and restaurants tables created in Task 3. To satisfy the four-table requirement, create delivery_agents and a capstone_orders table that links customers, restaurants, and delivery agents. Insert at least 10 rows into each of the four capstone tables before capturing the final outputs. The sample schema below includes foreign keys.

```
CREATE TABLE delivery_agents (
    agent_id INT PRIMARY KEY,
    agent_name VARCHAR(100) NOT NULL,
    city VARCHAR(60) NOT NULL,
    phone VARCHAR(20)
);

CREATE TABLE capstone_orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    agent_id INT NOT NULL,
    order_amount DECIMAL(10,2) NOT NULL,
    order_status VARCHAR(30) NOT NULL,
    order_date DATE NOT NULL,
    delivery_time_mins INT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),
    FOREIGN KEY (agent_id) REFERENCES delivery_agents(agent_id)
);
```

For a fully populated capstone dataset, extend the customer and restaurant seed data to at least 10 rows each, then add at least 10 delivery agents and 10 capstone orders. Use consistent IDs so every foreign key points to an existing record.

C1. Revenue report: restaurants with more than 3 orders

```
SELECT r.restaurant_id, r.name AS restaurant_name,
       COUNT(o.order_id) AS total_orders,
       SUM(o.order_amount) AS total_revenue,
       ROUND(AVG(o.order_amount), 2) AS average_order_value
FROM restaurants AS r
JOIN capstone_orders AS o ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(o.order_id) > 3
ORDER BY total_revenue DESC;
```

C2. Customer spend segmentation

```
SELECT c.customer_id, c.name AS customer_name,
       COALESCE(SUM(o.order_amount), 0) AS total_spend,
       CASE
           WHEN COALESCE(SUM(o.order_amount), 0) > 5000 THEN 'Platinum'
           WHEN COALESCE(SUM(o.order_amount), 0) >= 2000 THEN 'Gold'
           ELSE 'Standard'
       END AS customer_segment
FROM customers AS c
LEFT JOIN capstone_orders AS o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spend DESC;
```

C3. Top 3 restaurants by revenue in each city

```
WITH restaurant_totals AS (
    SELECT r.city, r.restaurant_id, r.name AS restaurant_name,
           SUM(o.order_amount) AS total_revenue
    FROM restaurants AS r
    JOIN capstone_orders AS o ON o.restaurant_id = r.restaurant_id
    GROUP BY r.city, r.restaurant_id, r.name
),
ranked_restaurants AS (
    SELECT *,
           DENSE_RANK() OVER (
               PARTITION BY city
               ORDER BY total_revenue DESC
           ) AS revenue_rank
    FROM restaurant_totals
)
SELECT city, restaurant_name, total_revenue, revenue_rank
FROM ranked_restaurants
WHERE revenue_rank <= 3
ORDER BY city, revenue_rank, restaurant_name;
```

C4. Most recent order for each customer

```
WITH customer_order_rank AS (
    SELECT o.*,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_date DESC, order_id DESC
           ) AS rn
    FROM capstone_orders AS o
)
SELECT c.customer_id, c.name AS customer_name,
       x.order_id, x.order_date, x.order_amount
FROM customers AS c
LEFT JOIN customer_order_rank AS x
       ON x.customer_id = c.customer_id
      AND x.rn = 1
ORDER BY c.customer_id;
```

C5. Create and query the daily order summary view

```
CREATE OR REPLACE VIEW vw_daily_order_summary AS
SELECT o.order_date,
       c.name AS customer_name,
       r.name AS restaurant_name,
       da.agent_name,
       o.order_amount,
       o.delivery_time_mins
FROM capstone_orders AS o
JOIN customers AS c ON c.customer_id = o.customer_id
JOIN restaurants AS r ON r.restaurant_id = o.restaurant_id
JOIN delivery_agents AS da ON da.agent_id = o.agent_id;

SELECT *
FROM vw_daily_order_summary
WHERE order_date >= CURRENT_DATE - INTERVAL 6 DAY
  AND order_date < CURRENT_DATE + INTERVAL 1 DAY
ORDER BY order_date DESC;
```

The date condition includes today and the six preceding calendar dates (seven dates total). If 'last 7 days' should mean the last 168 hours, use a DATETIME column and filter from NOW() - INTERVAL 7 DAY.

Section D — AI-Augmented Learning

D1. Exact prompt used

Write a MySQL 8+ query to find the top 3 delivery agents per city for a food delivery platform, ranked by number of completed orders. Include each agent's average delivery time, filter to agents with more than 5 completed orders in that city, and sort by city alphabetically then rank ascending. Assume tables delivery_agents(agent_id, agent_name, city) and capstone_orders(order_id, agent_id, order_status, delivery_time_mins).

D2. AI-generated original query

```
WITH agent_metrics AS (
    SELECT da.city, da.agent_id, da.agent_name,
           COUNT(o.order_id) AS completed_orders,
           AVG(o.delivery_time_mins) AS avg_delivery_time
    FROM delivery_agents AS da
    JOIN capstone_orders AS o ON o.agent_id = da.agent_id
    WHERE o.order_status = 'Completed'
    GROUP BY da.city, da.agent_id, da.agent_name
),
ranked AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY city
               ORDER BY completed_orders DESC
           ) AS agent_rank
    FROM agent_metrics
    WHERE completed_orders > 5
)
SELECT city, agent_name, completed_orders,
       ROUND(avg_delivery_time, 2) AS avg_delivery_time, agent_rank
FROM ranked
WHERE agent_rank <= 3
ORDER BY city, agent_rank;
```

Potential issue to test: the original query uses ROW_NUMBER(), which forces unique ranks when agents tie on completed orders. If ties should share a rank, RANK() or DENSE_RANK() is more appropriate. NULL delivery times are also excluded from AVG automatically, but it is useful to report whether an agent has any non-NULL delivery times.

D3. Corrected version

```
WITH agent_metrics AS (
    SELECT da.city, da.agent_id, da.agent_name,
           COUNT(o.order_id) AS completed_orders,
           AVG(o.delivery_time_mins) AS avg_delivery_time,
           COUNT(o.delivery_time_mins) AS deliveries_with_time
    FROM delivery_agents AS da
    JOIN capstone_orders AS o ON o.agent_id = da.agent_id
    WHERE o.order_status = 'Completed'
    GROUP BY da.city, da.agent_id, da.agent_name
),
eligible_agents AS (
    SELECT *
    FROM agent_metrics
    WHERE completed_orders > 5
),
ranked AS (
    SELECT *,
           RANK() OVER (
               PARTITION BY city
               ORDER BY completed_orders DESC
           ) AS agent_rank
    FROM eligible_agents
)
SELECT city, agent_name, completed_orders,
       ROUND(avg_delivery_time, 2) AS avg_delivery_time,
       deliveries_with_time, agent_rank
FROM ranked
WHERE agent_rank <= 3
ORDER BY city ASC, agent_rank ASC, agent_name ASC;
```

D4. Debugging note

I moved the completed-order threshold into a separate eligibility step before ranking, so ineligible agents do not affect rank positions. I replaced ROW_NUMBER() with RANK() so agents with equal completed-order counts can share a rank. I also added deliveries_with_time to expose cases where all delivery times are NULL; AVG ignores NULL values, so the count helps interpret the average correctly.

Conclusion

This assessment demonstrates core MySQL analytics techniques including filtering, sorting, joins, aggregation, HAVING, scalar subqueries, CASE WHEN, indexes, CTEs, window functions, and views. Before submission, run the script in MySQL 8.0 or later, capture screenshots of the actual outputs, and ensure the capstone has at least 10 rows per required table. Sample data and queries should be reviewed and adapted so the final submission reflects your own testing.